KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

Department Student Record Management System

Submitted by

BAGAVATHI V

25BCS106

Under the Guidance of

Mr. ARAVIND

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **II**

DECLARATION

I hereby declare that the project entitled "**Department Student Record Management System"** submitted in partial fulfilment of the requirements for the course **JAVA PROGRAMMING** is a **record of my original work** carried out under the guidance of **Mr. ARAVIND**, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

BAGAVATHI V

CERTIFICATE

This is to certify that the project entitled **"Department Student Record Management System"** is a **bona fide work** carried out by **BAGAVATHI V I "B"/2025**, **BSC COMPUTER SCIENCE**, **KG College of Arts & Science**, in partial fulfilment of the requirements for the course **JAVA PROGRAMMING**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: COIMBATORE

Date:10/04/2026

Submitted for the Viva-Voce Examination held on 10/04/2026

Guide:	**Head of the Department**
(Technical Trainer, KGCAS)	(Technical Head, KGCAS)
Mr. Aravind	Mr. Boopalan Sundaramoorthy

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	8
1.1	Objective	8
1.2	Overview	8
1.3	Features	9
2	**Program Analysis**	10
2.1	Problem Definition	10
2.2	Project Overview	10
2.3	Scopes and Limitations	11
3	**System Design**	12
3.1	Modules	12

4	**Methodology**	13
4.1	Problem Analysis	13
4.2	System Design	13
4.3	Program Development	14
4.4	Testing and Validation	14
4.5	Result and Documentation	14
4.6	Algorithm	13
4.7	Flowchart	15
5	**Source Code**	18
6	**Input and Output Screens**	22
7	**Discussion and Conclusion**	24

8	**References**	25

ABSTRACT

The **Department Student Record Management System** is a Java-based application developed to streamline the process of storing, managing, and retrieving student information within an academic department. Manual record-keeping often leads to inefficiency, errors, and difficulty in updating data. This system addresses those challenges by providing a digital solution that ensures accuracy, consistency, and quick access to student records.The system allows administrators and faculty to maintain comprehensive student profiles, including personal details, academic performance, and departmental information. With features such as search and retrieval, role-based access, and secure data handling, it minimizes redundancy and enhances reliability. By integrating Java's object-oriented programming principles with database connectivity (such as JDBC), the system ensures robustness, scalability, and ease of maintenance.Ultimately, this project improves departmental efficiency by reducing paperwork and enabling better decision-making through timely and accurate student data. It serves as a practical example of how technology can transform traditional academic processes, making them more efficient, secure, and adaptable to the growing needs of educational institutions.

1.INTRODUCTION

In today's academic environment, managing student records efficiently is a critical task for any department. Traditional methods of maintaining records through paper files or spreadsheets often lead to challenges such as data redundancy, difficulty in retrieval, and lack of security. As the number of students increases, these manual processes become even more time-consuming and error-prone. To overcome these limitations, computerized systems have become essential for ensuring accuracy, accessibility, and reliability in record management.

The Department Student Record Management System is developed using Java, a robust and platform-independent programming language. The system provides a structured way to store and manage student information, including personal details, academic performance, and departmental data. By leveraging Java's object-oriented features and database connectivity (such as JDBC), the application ensures scalability, security, and ease of maintenance. It also offers functionalities like quick search, record updates, and role-based access to safeguard sensitive information.

This project not only reduces paperwork but also enhances departmental efficiency by enabling faster decision-making based on accurate student data. It demonstrates how technology can transform traditional academic processes into modern, automated systems that are adaptable to the growing needs of educational institutions.

1.1 Objective

The primary objective of the **Department Student Record Management System** is to provide an efficient and reliable platform for storing, managing, and retrieving student information within an academic department. By replacing traditional manual methods with a computerized system, the project aims to reduce errors, minimize redundancy, and ensure quick access to accurate student data.

Another key objective is to enhance departmental productivity by offering features such as secure record handling, role-based access, and easy search functionality. This ensures that administrators, faculty, and staff can manage student details effectively while maintaining data integrity and confidentiality.

Finally, the system seeks to demonstrate the practical application of Java programming concepts, including object-oriented design and database connectivity. It provides a scalable solution that can adapt to the growing needs of educational institutions, ultimately supporting better academic decision-making and streamlined departmental operations.

1.2 Overview

The Department Student Record Management System is a software application developed in Java to simplify and automate the process of managing student information within an academic department. It eliminates the inefficiencies of manual record-keeping by providing a centralized platform where student details such as personal information, academic performance, and departmental data can be securely stored and easily accessed.

The system is designed with a user-friendly interface and robust backend support using Java's object-oriented programming principles and database connectivity (via JDBC or similar frameworks). It offers essential functionalities such as adding new student records, updating existing information, searching for specific students, and generating reports. Role-based access ensures that only authorized personnel can view or modify sensitive data, thereby maintaining confidentiality and integrity.

By integrating technology into departmental operations, this project enhances efficiency, reduces paperwork, and supports better academic decision-making. It is scalable to accommodate growing student populations and adaptable to the evolving needs of educational institutions, making it a practical and reliable solution for modern record management.

1.3 Features

Student Information Management

Allows storage of personal details, academic records, and departmental data in a structured format.

Add, Update, and Delete Records

Provides functionality to insert new student records, modify existing details, or remove outdated entries.

Search and Retrieval

Enables quick access to student information using unique identifiers such as roll number, name, or department.

Role-Based Access Control

Ensures that only authorized users (administrators, faculty, staff) can view or modify sensitive data.

Data Security and Integrity

Protects records from unauthorized access and maintains accuracy by reducing redundancy.

Report Generation

Facilitates the creation of academic and departmental reports for analysis and decision-making.

Scalability and Flexibility

Designed to handle increasing student populations and adapt to evolving departmental requirements.

User-Friendly Interface

Provides a simple and intuitive interface for easy navigation and efficient record management.

2.PROGRAM ANALYSIS

Program analysis is an important phase in software development where the problem is studied in detail and the requirements of the system are identified. This stage helps in understanding how the system should work and what features are required to solve the problem efficiently.

The analysis phase focuses on identifying the current issues in the existing system and proposing a computerized solution that improves efficiency and accuracy.

2.1 Problem Definition

Managing student records manually in academic departments often leads to inefficiencies, errors, and difficulties in maintaining accurate data. Traditional methods such as paper files or spreadsheets are time-consuming, prone to duplication, and lack proper security measures. As the number of students increases, retrieving specific information becomes challenging, and updating records requires significant effort.

Departments also face issues with data consistency and accessibility. Faculty and administrators often struggle to quickly obtain reliable information for academic decisions, performance tracking, or report generation. Without a centralized system, records may be scattered, incomplete, or outdated, which affects the overall efficiency of departmental operations.

The problem, therefore, is the absence of a robust, automated system that can securely store, manage, and retrieve student records while ensuring accuracy, scalability, and ease of use. Addressing this issue requires a computerized solution that integrates modern programming techniques and database management to streamline departmental record handling.

2.2 Project Overview

The **Department Student Record Management System** is a Java-based application designed to automate and simplify the process of managing student information within an academic department. It provides a centralized platform where student details such as personal information, academic performance, and departmental data can be securely stored, updated, and retrieved. By replacing traditional manual methods, the system ensures accuracy, efficiency, and reliability in record handling.

The project is built using Java's object-oriented programming principles, ensuring modularity, scalability, and ease of maintenance. The frontend interface, developed with Java Swing or JavaFX, offers a user-friendly environment for administrators and faculty, while the backend database connected via JDBC ensures secure and efficient data storage. Core functionalities include adding new records, updating existing details, searching for students, generating reports, and implementing role-based access control to safeguard sensitive information.

This system enhances departmental productivity by reducing paperwork, minimizing redundancy, and enabling faster decision-making through timely access to accurate student data. It demonstrates how technology can transform traditional academic processes into modern, automated solutions that are adaptable to the growing needs of educational institutions.

2.3 Scopes and Limitations

Provides a centralized platform for storing and managing student records.

Supports operations such as adding, updating, deleting, and searching student details.

Ensures secure handling of data with role-based access control.

Generates academic and departmental reports for analysis and decision-making.

Designed using Java's object-oriented principles with database connectivity (JDBC).

Scalable to handle increasing student populations and departmental requirements.

Offers a user-friendly interface for administrators, faculty, and staff.

3.SYSTEM DESIGN

System design is the stage where the structure and components of the system are planned before the actual implementation. In this phase, the system is divided into different modules, and the workflow of the application is defined. Proper system design helps in organizing the program efficiently and makes development easier.

The **Department Student Record Management System** is a modular application designed to manage student records efficiently. Users can add new student details, update existing information, delete outdated records, and quickly retrieve data, ensuring accuracy and security.

The system is divided into modules such as the **Student Module** for personal and academic details, the **Department Module** for organizing departmental data, the **Admin Module** for role-based access control, and the **Report Module** for generating academic reports.

Each module works together to provide a complete solution. Data entered through the interface is validated, stored securely in the database, and retrieved when needed. This modular design makes the system scalable, maintainable, and reliable for departmental record management.

3.1 Modules

1 Student Module

Handles personal details (name, roll number, contact info) and academic records (grades, courses, attendance).

2 Department Module

Organizes departmental information, categorizes students by department, and maintains department-specific data.

3 Admin Module

Provides role-based access control, manages users, and ensures security of sensitive student information.

4 Faculty/Staff Module

Allows faculty and staff to view or update student records relevant to their department.

5 Report Module

Generates academic reports, performance summaries, and departmental statistics for decision-making.

6 Database Module

Manages data storage, retrieval, and connectivity using JDBC, ensuring consistency and integrity of records.

4.METHODOLOGY

The system is developed through three main phases: **requirement analysis**, **design**, and **implementation**. In the requirement phase, the needs of the department are identified, such as adding, updating, deleting, and retrieving student records.

During the design phase, the application is structured into modules (Student, Department, Admin, Faculty/Staff, and Report) and built on a three-tier architecture with a user interface, business logic, and database connectivity via JDBC.

Finally, in the implementation phase, the modules are coded in Java, integrated with the database, and tested for accuracy, security, and efficiency. This ensures a reliable, scalable, and user-friendly system for managing student records.

4.1. Problem Analysis

Manual record-keeping in departments leads to errors, duplication, and difficulty in retrieving accurate student information. Paper files or spreadsheets are time-consuming, insecure, and inefficient as student numbers grow. Updating records and generating reports becomes cumbersome, and data consistency is often compromised.

The core problem is the lack of a centralized, automated system that ensures accuracy, security, and quick access to student records while supporting scalability for growing academic needs.

4.2. System Design

The **Department Student Record Management System** uses a three-tier architecture with a presentation layer (Java Swing/JavaFX interface), a business logic layer (Java classes for operations, validation, and role-based access), and a data layer (relational database via JDBC).

The system is divided into modules: **Student** (personal and academic details), **Department** (departmental data), **Admin** (user access and security), **Faculty/Staff** (controlled updates), and **Report** (academic and departmental reports).

Data flows from user input through the interface, processed by business logic, stored in the database, and retrieved as search results or reports. This modular design ensures scalability, maintainability, and security, making the system reliable for managing student records.

4.3.Program Development

The system was developed using Java with a structured approach. It began with analyzing manual record-keeping problems, followed by designing a three-tier architecture (presentation, business logic, data layer). Core modules—Student, Department, Admin, Faculty/Staff, and Reports—were implemented to handle operations like add, update, delete, search, and generate reports.

Validation checks ensured accuracy, while role-based access provided security. Testing confirmed reliability and efficiency. The result is a scalable, user-friendly application that automates student record management and improves departmental productivity.

4.4. Testing and Validation

Testing ensures that the program works correctly and produces the expected results.

Different test cases were used to check the functionality of the system.

Example Test Cases:

Test Type	Purpose	Activities
Unit Testing	Verify each module works correctly	Test Student, Department, Admin, Faculty/Staff, and Report modules individually
Integration Testing	Ensure modules interact smoothly	Check data flow between interface, business logic, and database
System Testing	Validate overall performance and functionality	Test complete system for operations, security, and role-based access
Validation	Ensure accuracy and integrity of data	Input checks, prevent duplicates, restrict

		unauthorized access
User Acceptance Testing	Confirm usability and reliability in real scenarios	Faculty and administrators test system for ease of use and accuracy

This phase ensures that the program runs without errors and produces accurate results.

4.5. Result and Documentation

The **Department Student Record Management System** successfully automates student record handling by allowing users to add, update, delete, and retrieve information quickly and securely. The modular design ensures smooth data flow between the interface, business logic, and database, while role-based access control maintains security. Reports can be generated efficiently, making the system reliable, scalable, and user-friendly for departmental use.

4.6. Algorithm

Start

Login/Authentication

- Verify user credentials (Admin, Faculty, Staff).
- Grant access based on role.

Display Main Menu

- Options: Add Student, Update Student, Delete Student, Search Student, Generate Report, Exit.

Add Student

- Input student details (ID, Name, Department, Courses, Grades).
- Validate input (no duplicates, required fields).
- Store record in database.

Update Student

- Search student by ID.
- Modify details as required.
- Save updated record in database.

Delete Student

- Search student by ID.
- Remove record from database.

Search Student

- Input search criteria (ID, Name, Department).
- Retrieve and display matching records.

Generate Report

- Select report type (Academic, Departmental, Performance).
- Fetch data from database.
- Display or export report.

Exit

- Close application safely.

End

4.7. Flowchart

```
┌──────────┐
│ Start    │
└──────────┘
     ↓
┌──────────────────┐
│ Login/Verify     │
│ Credentials      │
└──────────────────┘
     ↓
┌──────────────────────────┐
│ Display Main Menu        │
│ (Add, Update, Delete,    │
│ Search, Report, Exit)    │
└──────────────────────────┘
     ↓
┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐
┌──────────────────┐   ┌──────────────────┐
│ Add Student      │   │ Update Student   │   │ Delete Student   │   │ Search Student   │   │ Generate
Report │
└──────────────────┘   └──────────────────┘   └──────────────────┘
└──────────────────┘   └──────────────────┘
     ↓          ↓          ↓          ↓          ↓
  Validate & Save   Modify & Save   Remove Record   Retrieve Records   Display Report
     ↓          ↓          ↓          ↓          ↓
┌──────────┐
```

```
│  Return  │
│  to Menu │
└──────┬───────┘
       ↓
┌──────────────┐
│   Exit   │
└──────┬───────┘
       ↓
┌──────────────┐
│   End    │
└──────────────┘
```

5.SOURCE CODE

```
import java.util.*;

class Student {
    int id;
    String name;
    String department;
    double grade;

    Student(int id, String name, String department, double grade) {
        this.id = id;
        this.name = name;
        this.department = department;
        this.grade = grade;
    }

    @Override
    public String toString() {
        return "ID: " + id + ", Name: " + name + ", Dept: " + department + ", Grade: " + grade;
    }
}

public class StudentRecordSystem {
    static Map<Integer, Student> studentDB = new HashMap<>();
    static Scanner sc = new Scanner(System.in);

    public static void main(String[] args) {
        int choice;
        do {
            System.out.println("\n--- Department Student Record Management ---");
            System.out.println("1. Add Student");
            System.out.println("2. Update Student");
```

```
        System.out.println("3. Delete Student");
        System.out.println("4. Search Student");
        System.out.println("5. Display All Students");
        System.out.println("6. Exit");
        System.out.print("Enter choice: ");
        choice = sc.nextInt();

        switch (choice) {
            case 1: addStudent(); break;
            case 2: updateStudent(); break;
            case 3: deleteStudent(); break;
            case 4: searchStudent(); break;
            case 5: displayAll(); break;
            case 6: System.out.println("Exiting..."); break;
            default: System.out.println("Invalid choice!");
        }
    } while (choice != 6);
}

static void addStudent() {
    System.out.print("Enter ID: ");
    int id = sc.nextInt();
    sc.nextLine(); // consume newline
    System.out.print("Enter Name: ");
    String name = sc.nextLine();
    System.out.print("Enter Department: ");
    String dept = sc.nextLine();
    System.out.print("Enter Grade: ");
    double grade = sc.nextDouble();

    studentDB.put(id, new Student(id, name, dept, grade));
    System.out.println("Student added successfully!");
}
```

```
static void updateStudent() {
    System.out.print("Enter ID to update: ");
    int id = sc.nextInt();
    if (studentDB.containsKey(id)) {
        sc.nextLine();
        System.out.print("Enter New Name: ");
        String name = sc.nextLine();
        System.out.print("Enter New Department: ");
        String dept = sc.nextLine();
        System.out.print("Enter New Grade: ");
        double grade = sc.nextDouble();

        studentDB.put(id, new Student(id, name, dept, grade));
        System.out.println("Student updated successfully!");
    } else {
        System.out.println("Student not found!");
    }
}

static void deleteStudent() {
    System.out.print("Enter ID to delete: ");
    int id = sc.nextInt();
    if (studentDB.remove(id) != null) {
        System.out.println("Student deleted successfully!");
    } else {
        System.out.println("Student not found!");
    }
}

static void searchStudent() {
    System.out.print("Enter ID to search: ");
    int id = sc.nextInt();
```

```
        Student s = studentDB.get(id);
        if (s != null) {
            System.out.println("Found: " + s);
        } else {
            System.out.println("Student not found!");
        }
    }

    static void displayAll() {
        if (studentDB.isEmpty()) {
            System.out.println("No records available.");
        } else {
            System.out.println("--- Student Records ---");
            for (Student s : studentDB.values()) {
                System.out.println(s);
            }
        }
    }
}
```

6.INPUT AND OUTPUT SCREENS

Input & Output

Case 1: Add Student

Input:

Code

Enter choice: 1

Enter ID: 101

Enter Name: John

Enter Department: Computer Science

Enter Grade: 8.5

Output:

Code

Student added successfully!

Case 2: Display All Students

Input:

Code

Enter choice: 5

Output:

Code

--- Student Records ---

ID: 101, Name: John, Dept: Computer Science, Grade: 8.5

Case 3: Search Student

Input:

Code

Enter choice: 4

Enter ID to search: 101

Output:

Code

Found: ID: 101, Name: John, Dept: Computer Science, Grade: 8.5

Case 4: Update Student

Input:

Code

Enter choice: 2

Enter ID to update: 101

Enter New Name: John Smith

Enter New Department: IT

Enter New Grade: 9.0

Output:

Code

Student updated successfully!

Case 5: Delete Student

Input:

Code

Enter choice: 3

Enter ID to delete: 101

Output:

Code

Student deleted successfully!

Case 6: Exit

Input:

Code

Enter choice: 6

Output:

Code

Exiting...

7.DISCUSSION & CONCLUSION

7.1 Discussion

The project demonstrates how a three-tier architecture can effectively manage departmental student records. By separating the presentation, business logic, and data layers, the system ensures clarity, maintainability, and scalability. Each module—Student, Department, Admin, Faculty/Staff, and Report—plays a distinct role in streamlining operations such as adding, updating, deleting, and retrieving records. Testing and validation confirmed that the system performs reliably, with secure role-based access and accurate data handling. Compared to manual methods, the system reduces errors, saves time, and improves accessibility of information for administrators and faculty.

7.2 Conclusion

The **Department Student Record Management System** successfully addresses the challenges of manual record-keeping by providing an automated, secure, and user-friendly solution. It ensures efficient data management, quick retrieval of information, and reliable reporting. The modular design and layered architecture make the system scalable for future enhancements, while validation ensures accuracy and integrity of records. Overall, the project achieves its goal of improving departmental productivity and decision-making through effective student record management.

8.REFERENCES

The following resources were used during the development of this project:

Java Documentation – Oracle. *The Java™ Tutorials*. Available at: https://docs.oracle.com/javase/tutorial/

JDBC Guide – Oracle. *Java Database Connectivity (JDBC) Overview*. Available at: https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc/ (docs.oracle.com in Bing)

Sommerville, I. (2016). *Software Engineering* (10th Edition). Pearson Education.

Pressman, R. S. (2014). *Software Engineering: A Practitioner's Approach* (8th Edition). McGraw-Hill.

W3Schools. *Java Programming Tutorial*. Available at: https://www.w3schools.com/java/

GeeksforGeeks. *Java and JDBC Tutorials*. Available at: https://www.geeksforgeeks.org/java/